

```

import { q, one } from '../db.js';
import { resolveAll } from './providers.js';

// The router does not know any vendor. It asks every bound model provider what
// it has, scores the candidates against the task, and picks. If the pick fails
// it falls back to the next one and records that it did.
//
// A task looks like: { need: 'reasoning' | 'cheap' | 'fast' | 'vision', maxCents

export async function route({ ctx, task, executionId = null }) {
  const providers = await resolveAll({ slot: 'model', businessId: ctx.businessId
  if (providers.length === 0) throw new Error('no model provider bound');

  const candidates = [];
  for (const p of providers) {
    for (const m of await p.module.models(p.config)) {
      candidates.push({ ...m, packageKey: p.manifest.key, provider: p });
    }
  }
  if (candidates.length === 0) throw new Error('no models offered by any bound pr

  const scored = candidates
    .map(c => ({ c, score: score(c, task) }))
    .filter(x => x.score > -Infinity)
    .sort((a, b) => b.score - a.score);

  if (scored.length === 0) throw new Error('no model satisfies the task constrain

  let fellBackFrom = null;
  for (const { c } of scored) {
    const started = Date.now();
    try {
      const out = await c.provider.module.complete({ model: c.id, task, config: c
      const latency = Date.now() - started;

      await one(
        `insert into routing_decision
          (business_id, execution_id, slot_key, candidates, chose, reason, fell_
          values ($1,$2,'model',$3::jsonb,$4,$5,$6,$7,$8) returning *`,
        [ctx.businessId, executionId,
        JSON.stringify(scored.slice(0, 6).map(s => ({ model: s.c.id, from: s.c.p
        `${c.packageKey}:${c.id}`, reasonFor(task), fellBackFrom, latency,
        estimateCents(c, out)]
      );

```

```

        return { model: c.id, from: c.packageKey, output: out.text ?? out, latencyM
    } catch (e) {
        fellBackFrom = `${c.packageKey}:${c.id}`;
    }
}
throw new Error('every candidate model failed');
}

function score(model, task) {
    if (task.maxCents !== undefined && model.centsPerCall > task.maxCents) return -
    if (task.need === 'vision' && !model.vision) return -Infinity;
    if (task.local === true && !model.local) return -Infinity;

    const weights = {
        reasoning: { strength: 3, speed: 0.5, cheapness: 0.5 },
        cheap:     { strength: 0.5, speed: 1, cheapness: 3 },
        fast:      { strength: 0.5, speed: 3, cheapness: 1 },
        vision:    { strength: 2, speed: 1, cheapness: 1 },
    }[task.need] ?? { strength: 1, speed: 1, cheapness: 1 };

    const cheapness = model.centsPerCall > 0 ? 1 / model.centsPerCall : 10;
    return weights.strength * (model.strength ?? 1)
        + weights.speed * (model.speed ?? 1)
        + weights.cheapness * Math.min(cheapness, 10);
}

function reasonFor(task) {
    return `need=${task.need ?? 'balanced'}` +
        (task.maxCents !== undefined ? ` maxCents=${task.maxCents}` : '') +
        (task.local ? ' local-only' : '');
}

function estimateCents(model, out) {
    return out.cents ?? model.centsPerCall ?? 0;
}

export async function decisions(businessId, limit = 50) {
    return q(
        `select slot_key, chose, reason, fell_back_from, latency_ms, cost_cents, crea
        from routing_decision where business_id = $1 order by created_at desc limi
        [businessId]
    );
}

```